

心脏+获取苹果手表健康数据-详细技术设计文档

以下是基于苹果手表健康数据的MySQL建表语句，涵盖步数、心率、睡眠（时段和阶段）、运动、血氧、呼吸率、体温和HRV（心率变异性，用于推测情绪压力）。所有数据均限于苹果手表实际支持的功能（截至2025年3月12日）。

一、获取数据说明

第一次获取时长

6个月

业务形态

放到助手：快捷键 或 主动推送

助手菜单：消息红点逻辑要做

数据来源

- 步数、心率、血氧**：通过内置传感器实时测量。
- 睡眠**：包括总体时段和阶段（清醒、浅睡、深睡、REM），由加速度计和心率传感器推导。
- 运动**：记录类型、距离和卡路里，用户手动开始或自动检测。
- 呼吸率**：睡眠期间通过加速度计测量（watchOS 8及以上）。
- 体温**：睡眠期间手腕温度（Series 8及以上）。
- HRV**：通过光学心率传感器计算SDNN值，可间接推测情绪压力。

字段设计考量

- 时间戳**：使用 `DATETIME(6)` 支持微秒精度，适合高频数据如ECG。
- 单位**：明确标注（如心率bpm、血氧%、体温°C），便于查询和分析。
- 外键**：通过 `manager_no` 关联用户，确保数据完整性。
- 可空字段**：`distance` 和 `calories_burned` 在运动表中可为空，适应不同运动类型。

优化建议

- 为频繁查询字段（如 `manager_no`、`timestamp`）添加索引。

苹果手表数据获取策略

以下是关于如何获取苹果手表健康数据的策略，结合其支持的数据类型（如步数、心率、睡眠、运动、ECG、血氧、呼吸率、体温和HRV）和实际应用场景。所有内容基于苹果手表现有功能（截至2025年3月12日），并提供技术实现建议。

1. 数据类型与来源

苹果手表通过内置传感器和HealthKit框架收集多种健康数据，具体包括：

- 步数**：通过加速度计统计，记录时间段内的累计步数。
- 心率**：光学心率传感器实时测量，单位为bpm。
- 睡眠**：包括总体时段和阶段（清醒、浅睡、深睡、REM），由加速度计和心率数据推导。
- 运动**：记录类型（如跑步、游泳）、距离和卡路里，通过用户手动开始或自动检测。
- 血氧**：红外传感器测量血氧饱和度，单位为百分比。
- 呼吸率**：睡眠期间通过加速度计测量，单位为每分钟呼吸次数。
- 体温**：睡眠期间手腕温度（Series 8及以上），单位为摄氏度。
- HRV**：心率变异性（SDNN值，毫秒），可用于推测情绪压力。

这些数据存储在iPhone的Health app中，开发者可通过HealthKit API访问。

2. 数据获取策略

2.1 通过HealthKit API访问

- **前提：**需要用户授权访问HealthKit数据。
- **步骤：**
 - 配置权限：**在Xcode中启用HealthKit，添加 `Info.plist` 权限描述（如 `NSHealthShareUsageDescription`）。
 - 初始化HealthKit：**创建 `HKHealthStore` 实例。
 - 请求授权：**

代码块

```
1 let healthStore = HKHealthStore()
2 let typesToRead: Set = [
3     HKObjectType.quantityType(forIdentifier: .heartRate)!,
4     HKObjectType.quantityType(forIdentifier: .stepCount)!,
5     HKObjectType.quantityType(forIdentifier: .bloodOxygen)!,
6     HKObjectType.quantityType(forIdentifier: .respiratoryRate)!,
7     HKObjectType.quantityType(forIdentifier: .bodyTemperature)!,
8     HKObjectType.quantityType(forIdentifier: .heartRateVariabilitySDNN)!
9 ]
10 healthStore.requestAuthorization(toShare: nil, read: typesToRead) {
11     success, error in
12     if success {
13         print("授权成功")
14     } else {
15         print("授权失败: \(error?.localizedDescription ?? "")")
16     }
17 }
```

- 查询数据：**以心率为例，获取最近24小时数据：

代码块

```
1 let heartRateType = HKQuantityType.quantityType(forIdentifier:
2     .heartRate)!
3 let now = Date()
4 let startDate = Calendar.current.date(byAdding: .hour, value: -24, to:
5     now)!
6 let predicate = HKQuery.predicateForSamples(withStart: startDate, end:
7     now, options: .strictStartDate)
8 let query = HKSampleQuery(sampleType: heartRateType, predicate:
9     predicate, limit: HKObjectQueryNoLimit, sortDescriptors: nil) { query,
10     samples, error in
```

```

6      guard let samples = samples as? [HKQuantitySample], error == nil else
      {
7          print("查询失败: \(error?.localizedDescription ?? "")")
8          return
9      }
10     for sample in samples {
11         let heartRate = sample.quantity.doubleValue(for: HKUnit(from:
            "count/min"))
12         let date = sample.startDate
13         print("心率: \(heartRate) bpm, 时间: \(date)")
14     }
15 }
16 healthStore.execute(query)

```

• 注意事项:

- ECG数据需查询 `HKObjectType.electrocardiogramType()`，并单独处理序列点。
- 睡眠数据需查询 `HKCategoryType.categoryType(forIdentifier: .sleepAnalysis)`，解析阶段。
- 运动数据通过 `HKWorkout` 类型获取。

2.2 实时数据订阅 - 优先不考虑

- 场景：需要**实时监控心率或血氧**。
- 实现：使用 `HKObserverQuery` 订阅数据更新：

代码块

```

1  let heartRateType = HKQuantityType.quantityType(forIdentifier: .heartRate)!
2  let observerQuery = HKObserverQuery(sampleType: heartRateType, predicate:
    nil) { query, completionHandler, error in
3      if error == nil {
4          print("心率数据更新")
5          // 执行具体查询
6          completionHandler()
7      }
8  }
9  healthStore.execute(observerQuery)

```

二、接口设计

上报数据类型：Type

类型 type	功能	请求参数	响应
step-count	上传步数数据	StepCountDTO (JSON)	StepCount (JSON)
heart-rate	上传心率数据	HeartRateDTO (JSON)	HeartRate (JSON)
sleep-period	上传睡眠时段数据	SleepPeriodDTO (JSON)	SleepPeriod (JSON)
sleep-stage	上传睡眠阶段数据	SleepStageDTO (JSON)	SleepStage (JSON)
exercise	上传运动数据	ExerciseDTO (JSON)	Exercise (JSON)
blood-oxygen	上传血氧数据	BloodOxygenDTO (JSON)	BloodOxygen (JSON)
respiration-rate	上传呼吸率数据	RespirationRateDTO (JSON)	RespirationRate (JSON)
body-temperature	上传体温数据	BodyTemperatureDTO (JSON)	BodyTemperature (JSON)
hrv	上传HRV数据	HRVDTO (JSON)	HRV (JSON)

将所有数据类型合并为一个接口

- `POST /api/health/data -> /mini/api`
- 请求体包含数据类型标识（如 `type` 字段）和对应的数据。

示例请求：

我们接口json改造下

Header

body

上传： 太长考虑压缩 或 分次

代码块

```
1  [
2    {
3      "type": "step-count",
4      "data": {
5        "managerNo": 1,
6        "startTime": "2025-03-12T08:00:00",
7        "endTime": "2025-03-12T08:10:00",
8        "steps": 500
9      }
10   },
11   {
12     "type": "step-count",
13     "data": {
14       "managerNo": 1,
15       "startTime": "2025-03-12T08:05:00",
16       "endTime": "2025-03-12T08:13:00",
17       "steps": 500
18     }
19   },
20   {
21     "type": "heart-rate",
22     "data": {
23       "managerNo": 1,
24       "timestamp": "2025-03-12T08:05:00",
25       "value": 75.5
26     }
27   }
28 ]
```

问题：每次打开获取数据逻辑，去重

上报接口

1. 技术实现

- 优点：
 - **统一入口**：单一接口减少了Controller层的代码量，客户端只需记住一个端点。

- **动态解析**：通过 `type` 字段和Jackson的 `@JsonTypeInfo` 注解，可动态解析不同数据类型的DTO。
- **批量支持**：支持List格式的请求，便于一次上传多种数据。

- **实现示例：**

代码块

```
1 package com.example.health.dto;
2
3 import com.fasterxml.jackson.annotation.JsonSubTypes;
4 import com.fasterxml.jackson.annotation.JsonTypeInfo;
5 import lombok.Data;
6
7 @Data
8 public class HealthDataRequest {
9     private String type; // 数据类型，如"step-count", "heart-rate"
10
11     @JsonTypeInfo(use = JsonTypeInfo.Id.NAME, include =
12         JsonTypeInfo.As.EXTERNAL_PROPERTY, property = "type")
13     @JsonSubTypes({
14         @JsonSubTypes.Type(value = StepCountDTO.class, name = "step-count"),
15         @JsonSubTypes.Type(value = HeartRateDTO.class, name = "heart-rate"),
16         @JsonSubTypes.Type(value = SleepPeriodDTO.class, name = "sleep-
17             period"),
18         // 其他类型...
19     })
20     private Object data; // 具体数据
21 }
```

代码块

```
1 package com.example.health.controller;
2
3 import com.example.health.dto.HealthDataRequest;
4 import com.example.health.service.*;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.*;
8
9 @RestController
10 @RequestMapping("/api/health")
11 public class HealthController {
```

```

12
13     @Autowired
14     private StepCountService stepCountService;
15     @Autowired
16     private HeartRateService heartRateService;
17     // 其他服务...
18
19     @PostMapping("/data")
20     public ResponseEntity<?> uploadHealthData(@RequestBody HealthDataRequest
request) {
21         switch (request.getType()) {
22             case "step-count":
23                 return
ResponseEntity.ok(stepCountService.saveStepCount((StepCountDTO)
request.getData()));
24             case "heart-rate":
25                 return
ResponseEntity.ok(heartRateService.saveHeartRate((HeartRateDTO)
request.getData()));
26             // 其他case...
27             default:
28                 return ResponseEntity.badRequest().body("未知的数据类型: " +
request.getType());
29         }
30     }
31 }

```

- **可行性：**技术上完全可行，Spring Boot支持动态类型解析和批量处理。

2. 性能

- **优点：**

- **批量上传：**合并接口天然支持批量操作，减少网络请求次数，提升效率。
- **事务管理：**可在一个事务中处理多种数据，保证一致性。

- **缺点：**

- **复杂逻辑：**单一接口需处理所有类型，逻辑分支多，可能增加处理时间。
- **资源占用：**批量上传大数据（如ECG数据点）可能导致内存压力。

- **评估：**性能影响取决于数据量和请求频率。少量数据时合并更高效，大数据需优化（如分片上传）。

3. 灵活性

- **优点：**
 - **扩展性：**新增数据类型只需更新DTO和Service，无需修改接口路径。
- **缺点：**
 - **客户端复杂性：**客户端需正确构造请求体，增加了开发负担。
 - **错误定位：**单一接口出错时，定位具体数据类型问题较困难。
- **评估：**灵活性较高，但客户端需配合调整。

4. 维护性

- **优点：**
 - **代码简洁：**减少Controller数量，路由管理更简单。
- **缺点：**
 - **逻辑集中：**所有逻辑集中在一个方法，维护时需小心避免影响其他类型。
 - **文档复杂：**API文档需详细描述每种 `type` 的格式，增加维护成本。
- **评估：**短期维护成本降低，长期可能因逻辑复杂性增加维护难度。

5. 实际应用场景

- **适用场景：**
 - **批量同步：**如HealthKit导出多种数据后一次性上传。
 - **简单客户端：**客户端希望简化调用逻辑。
- **不适用场景：**
 - **实时上传：**如心率实时监控，单一接口可能增加延迟。
 - **独立模块：**不同团队负责不同数据类型时，独立接口更清晰。
- **评估：**合并接口适合批量或集中式上传，不适合高频实时场景。

三、MySQL建表语句

步数表 (Step_Count)

代码块

```
1 CREATE TABLE Step_Count (  
2     id INT PRIMARY KEY AUTO_INCREMENT COMMENT '步数记录ID, 主键, 自增',  
3     manager_no INT NOT NULL COMMENT '用户ID, 外键',  
4     start_time DATETIME(6) NOT NULL COMMENT '开始时间, 精确到微秒',  
5     end_time DATETIME(6) NOT NULL COMMENT '结束时间, 精确到微秒',  
6     steps INT NOT NULL COMMENT '步数, 整数, 表示该时间段内的总步数'  
7 );
```

心率表 (Heart_Rate)

代码块

```
1 CREATE TABLE Heart_Rate (  
2     id INT PRIMARY KEY AUTO_INCREMENT COMMENT '心率记录ID, 主键, 自增',  
3     manager_no INT NOT NULL COMMENT '用户ID, 外键',  
4     timestamp DATETIME(6) NOT NULL COMMENT '时间戳, 测量时间, 精确到微秒',  
5     value TEXT NOT NULL COMMENT '心率值, 单位: 每天心跳数组 (bpm) '  
6 );
```

睡眠时段表 (Sleep_Periods)

代码块

```
1 CREATE TABLE Sleep_Periods (  
2     id INT PRIMARY KEY AUTO_INCREMENT COMMENT '睡眠时段ID, 主键, 自增',  
3     manager_no INT NOT NULL COMMENT '用户ID, 外键',  
4     start_time DATETIME(6) NOT NULL COMMENT '开始时间, 精确到微秒',  
5     end_time DATETIME(6) NOT NULL COMMENT '结束时间, 精确到微秒',  
6     total_sleep_duration INT COMMENT '总睡眠时长, 单位: 分钟, 可为空'  
7  
8 );
```

睡眠阶段类型表 (Sleep_Stages_Type) 配置表 - 枚举

代码块

```
1 CREATE TABLE Sleep_Stages_Type (  
2     id INT PRIMARY KEY AUTO_INCREMENT COMMENT '睡眠阶段类型ID, 主键, 自增',  
3     stage_name VARCHAR(20) NOT NULL UNIQUE COMMENT '阶段名称, 唯一, 如“清醒”、“浅睡”、“深睡”、“REM”'  
4 );
```

睡眠阶段表 (Sleep_Stages)

代码块

```
1 CREATE TABLE Sleep_Stages (  
2     id INT PRIMARY KEY AUTO_INCREMENT COMMENT '睡眠阶段记录ID, 主键, 自增',  
3     sleep_period_id INT NOT NULL COMMENT '睡眠时段ID, 外键, 关联Sleep_Periods表',  
4     stage_id INT NOT NULL COMMENT '阶段类型ID, 外键, 关联Sleep_Stages_Type表',  
5     start_time DATETIME(6) NOT NULL COMMENT '开始时间, 精确到微秒',  
6     end_time DATETIME(6) NOT NULL COMMENT '结束时间, 精确到微秒'  
7 );
```

运动类型表 (Exercise_Types)

代码块

```
1 CREATE TABLE Exercise_Types (  
2     id INT PRIMARY KEY AUTO_INCREMENT COMMENT '运动类型ID, 主键, 自增',  
3     type_name VARCHAR(50) NOT NULL UNIQUE COMMENT '运动类型名称, 唯一, 如“跑步”、“骑行”'  
4 );
```

运动表 (Exercise)

代码块

```
1 CREATE TABLE Exercise (  
2     exercise_id INT PRIMARY KEY AUTO_INCREMENT COMMENT '运动记录ID, 主键, 自增',  
3     manager_no INT NOT NULL COMMENT '用户ID, 外键',  
4     exercise_type_id INT NOT NULL COMMENT '运动类型ID, 外键, 关联Exercise_Types表',
```

```
5 start_time DATETIME(6) NOT NULL COMMENT '开始时间, 精确到微秒',
6 end_time DATETIME(6) NOT NULL COMMENT '结束时间, 精确到微秒',
7 distance FLOAT COMMENT '距离, 单位: 公里, 可为空',
8 calories_burned INT COMMENT '消耗卡路里, 单位: 卡, 可为空',
9 FOREIGN KEY (exercise_type_id) REFERENCES Exercise_Types(exercise_type_id)
COMMENT '外键约束, 关联Exercise_Types表'
10 );
```

血氧表 (Blood_Oxygen)

代码块

```
1 CREATE TABLE Blood_Oxygen (
2     id INT PRIMARY KEY AUTO_INCREMENT COMMENT '血氧记录ID, 主键, 自增',
3     manager_no INT NOT NULL COMMENT '用户ID, 外键',
4     timestamp DATETIME(6) NOT NULL COMMENT '时间戳, 测量时间, 精确到微秒',
5     value FLOAT NOT NULL COMMENT '血氧饱和度, 单位: 百分比 (%) '
6
7 );
```

呼吸率表 (Respiration_Rate)

代码块

```
1 CREATE TABLE Respiration_Rate (
2     id INT PRIMARY KEY AUTO_INCREMENT COMMENT '呼吸率记录ID, 主键, 自增',
3     manager_no INT NOT NULL COMMENT '用户ID, 外键',
4     timestamp DATETIME(6) NOT NULL COMMENT '时间戳, 测量时间, 精确到微秒',
5     value FLOAT NOT NULL COMMENT '呼吸率, 单位: 每分钟呼吸次数 (breaths/min) '
6
7 );
```

体温表 (Body_Temperature)

代码块

```
1 CREATE TABLE Body_Temperature (
2     id INT PRIMARY KEY AUTO_INCREMENT COMMENT '体温记录ID, 主键, 自增',
3     manager_no INT NOT NULL COMMENT '用户ID, 外键',
```

```
4 timestamp DATETIME(6) NOT NULL COMMENT '时间戳，测量时间，精确到微秒'，
5 value FLOAT NOT NULL COMMENT '手腕温度，单位：摄氏度（°C）'
6
7 );
```

HRV表 (HRV)

代码块

```
1 CREATE TABLE HRV (
2     id INT PRIMARY KEY AUTO_INCREMENT COMMENT 'HRV记录ID，主键，自增'，
3     manager_no INT NOT NULL COMMENT '用户ID，外键'，
4     timestamp DATETIME(6) NOT NULL COMMENT '时间戳，测量时间，精确到微秒'，
5     value FLOAT NOT NULL COMMENT '心率变异性值，单位：毫秒（ms），SDNN值，可用于推测
    情绪压力'
6
7 );
```

四、Java 服务设计

1. 实体类 (Entity)

步数实体 (Step_Count)

代码块

```
1 package com.example.health.entity;
2
3 import jakarta.persistence.*;
4 import lombok.Data;
5 import java.time.LocalDateTime;
6
7 @Entity
8 @Table(name = "Step_Count")
9 @Data
10 public class StepCount {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     @Column(name = "step_count_id")
14     private Integer stepCountId; // 步数记录ID，主键，自增
15 }
```

```

16 @ManyToOne
17 @JoinColumn(name = "manager_no", nullable = false)
18 private User user; // 用户ID, 外键
19
20 @Column(name = "start_time", nullable = false)
21 private LocalDateTime startTime; // 开始时间, 精确到微秒
22
23 @Column(name = "end_time", nullable = false)
24 private LocalDateTime endTime; // 结束时间, 精确到微秒
25
26 @Column(name = "steps", nullable = false)
27 private Integer steps; // 步数, 整数, 通过加速度计统计, 记录时间段内的累计步数
28 }

```

心率实体 (Heart_Rate)

代码块

```

1 package com.example.health.entity;
2
3 import jakarta.persistence.*;
4 import lombok.Data;
5 import java.time.LocalDateTime;
6
7 @Entity
8 @Table(name = "Heart_Rate")
9 @Data
10 public class HeartRate {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     @Column(name = "heart_rate_id")
14     private Integer heartRateId; // 心率记录ID, 主键, 自增
15
16     @ManyToOne
17     @JoinColumn(name = "manager_no", nullable = false)
18     private User user; // 用户ID, 外键
19
20     @Column(name = "timestamp", nullable = false)
21     private LocalDateTime timestamp; // 时间戳, 测量时间, 精确到微秒
22
23     @Column(name = "value", nullable = false)
24     private String value; // 心率值, 单位: 每分钟心跳次数 (bpm), 光学心率传感器实时测
25     量
26 }

```

睡眠时段实体 (Sleep_Periods)

代码块

```
1 package com.example.health.entity;
2
3 import jakarta.persistence.*;
4 import lombok.Data;
5 import java.time.LocalDateTime;
6
7 @Entity
8 @Table(name = "Sleep_Periods")
9 @Data
10 public class SleepPeriod {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     @Column(name = "sleep_period_id")
14     private Integer sleepPeriodId; // 睡眠时段ID, 主键, 自增
15
16     @ManyToOne
17     @JoinColumn(name = "manager_no", nullable = false)
18     private User user; // 用户ID, 外键
19
20     @Column(name = "start_time", nullable = false)
21     private LocalDateTime startTime; // 开始时间, 精确到微秒
22
23     @Column(name = "end_time", nullable = false)
24     private LocalDateTime endTime; // 结束时间, 精确到微秒
25
26     @Column(name = "total_sleep_duration")
27     private Integer totalSleepDuration; // 总睡眠时长, 单位: 分钟, 由加速度计和心率
    数据推导
28 }
```

睡眠阶段类型实体 (Sleep_Stages_Type)

代码块

```
1 package com.example.health.entity;
2
3 import jakarta.persistence.*;
4 import lombok.Data;
```

```

5
6 @Entity
7 @Table(name = "Sleep_Stages_Type")
8 @Data
9 public class SleepStageType {
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     @Column(name = "stage_id")
13     private Integer stageId; // 睡眠阶段类型ID, 主键, 自增
14
15     @Column(name = "stage_name", nullable = false, unique = true, length = 20)
16     private String stageName; // 阶段名称, 唯一, 如“清醒”、“浅睡”、“深睡”、“REM”
17 }

```

睡眠阶段实体 (Sleep_Stages)

代码块

```

1 package com.example.health.entity;
2
3 import jakarta.persistence.*;
4 import lombok.Data;
5 import java.time.LocalDateTime;
6
7 @Entity
8 @Table(name = "Sleep_Stages")
9 @Data
10 public class SleepStage {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     @Column(name = "sleep_stage_id")
14     private Integer sleepStageId; // 睡眠阶段记录ID, 主键, 自增
15
16     @ManyToOne
17     @JoinColumn(name = "sleep_period_id", nullable = false)
18     private SleepPeriod sleepPeriod; // 睡眠时段ID, 外键, 关联Sleep_Periods表
19
20     @ManyToOne
21     @JoinColumn(name = "stage_id", nullable = false)
22     private SleepStageType stageType; // 阶段类型ID, 外键, 关联Sleep_Stages_Type表
23
24     @Column(name = "start_time", nullable = false)
25     private LocalDateTime startTime; // 开始时间, 精确到毫秒
26

```



```
27 @Column(name = "end_time", nullable = false)
28 private LocalDateTime endTime; // 结束时间, 精确到微秒
29 }
```

运动类型实体 (Exercise_Types)

代码块

```
1 package com.example.health.entity;
2
3 import jakarta.persistence.*;
4 import lombok.Data;
5
6 @Entity
7 @Table(name = "Exercise_Types")
8 @Data
9 public class ExerciseType {
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     @Column(name = "exercise_type_id")
13     private Integer exerciseTypeId; // 运动类型ID, 主键, 自增
14
15     @Column(name = "type_name", nullable = false, unique = true, length = 50)
16     private String typeName; // 运动类型名称, 唯一, 如“跑步”、“游泳”
17 }
```

运动实体 (Exercise)

代码块

```
1 package com.example.health.entity;
2
3 import jakarta.persistence.*;
4 import lombok.Data;
5 import java.time.LocalDateTime;
6
7 @Entity
8 @Table(name = "Exercise")
9 @Data
10 public class Exercise {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```

13 @Column(name = "exercise_id")
14 private Integer exerciseId; // 运动记录ID, 主键, 自增
15
16 @ManyToOne
17 @JoinColumn(name = "manager_no", nullable = false)
18 private User user; // 用户ID, 外键
19
20 @ManyToOne
21 @JoinColumn(name = "exercise_type_id", nullable = false)
22 private ExerciseType exerciseType; // 运动类型ID, 外键, 关联Exercise_Types表
23
24 @Column(name = "start_time", nullable = false)
25 private LocalDateTime startTime; // 开始时间, 精确到微秒
26
27 @Column(name = "end_time", nullable = false)
28 private LocalDateTime endTime; // 结束时间, 精确到微秒
29
30 @Column(name = "distance")
31 private Float distance; // 距离, 单位: 公里, 可为空
32
33 @Column(name = "calories_burned")
34 private Integer caloriesBurned; // 消耗卡路里, 单位: 卡, 可为空, 通过用户手动开始
    或自动检测
35 }

```

血氧实体 (Blood_Oxygen)

代码块

```

1 package com.example.health.entity;
2
3 import jakarta.persistence.*;
4 import lombok.Data;
5 import java.time.LocalDateTime;
6
7 @Entity
8 @Table(name = "Blood_Oxygen")
9 @Data
10 public class BloodOxygen {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     @Column(name = "blood_oxygen_id")
14     private Integer bloodOxygenId; // 血氧记录ID, 主键, 自增
15

```

```

16 @ManyToOne
17 @JoinColumn(name = "manager_no", nullable = false)
18 private User user; // 用户ID, 外键
19
20 @Column(name = "timestamp", nullable = false)
21 private LocalDateTime timestamp; // 时间戳, 测量时间, 精确到微秒
22
23 @Column(name = "value", nullable = false)
24 private Float value; // 血氧饱和度, 单位: 百分比 (%), 红外传感器测量
25 }

```

呼吸率实体 (Respiration_Rate)

代码块

```

1 package com.example.health.entity;
2
3 import jakarta.persistence.*;
4 import lombok.Data;
5 import java.time.LocalDateTime;
6
7 @Entity
8 @Table(name = "Respiration_Rate")
9 @Data
10 public class RespirationRate {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     @Column(name = "respiration_rate_id")
14     private Integer respirationRateId; // 呼吸率记录ID, 主键, 自增
15
16     @ManyToOne
17     @JoinColumn(name = "manager_no", nullable = false)
18     private User user; // 用户ID, 外键
19
20     @Column(name = "timestamp", nullable = false)
21     private LocalDateTime timestamp; // 时间戳, 测量时间, 精确到微秒
22
23     @Column(name = "value", nullable = false)
24     private Float value; // 呼吸率, 单位: 每分钟呼吸次数 (breaths/min), 睡眠期间通过
    加速度计测量
25 }

```

体温实体 (Body_Temperature)

代码块

```
1 package com.example.health.entity;
2
3 import jakarta.persistence.*;
4 import lombok.Data;
5 import java.time.LocalDateTime;
6
7 @Entity
8 @Table(name = "Body_Temperature")
9 @Data
10 public class BodyTemperature {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     @Column(name = "body_temperature_id")
14     private Integer bodyTemperatureId; // 体温记录ID, 主键, 自增
15
16     @ManyToOne
17     @JoinColumn(name = "manager_no", nullable = false)
18     private User user; // 用户ID, 外键
19
20     @Column(name = "timestamp", nullable = false)
21     private LocalDateTime timestamp; // 时间戳, 测量时间, 精确到微秒
22
23     @Column(name = "value", nullable = false)
24     private Float value; // 手腕温度, 单位: 摄氏度 (°C), 睡眠期间测量 (Series 8及以上)
25 }
```

HRV实体 (HRV)

代码块

```
1 package com.example.health.entity;
2
3 import jakarta.persistence.*;
4 import lombok.Data;
5 import java.time.LocalDateTime;
6
7 @Entity
8 @Table(name = "HRV")
9 @Data
10 public class HRV {
```

```
11 @Id
12 @GeneratedValue(strategy = GenerationType.IDENTITY)
13 @Column(name = "hrv_id")
14 private Integer hrvId; // HRV记录ID, 主键, 自增
15
16 @ManyToOne
17 @JoinColumn(name = "manager_no", nullable = false)
18 private User user; // 用户ID, 外键
19
20 @Column(name = "timestamp", nullable = false)
21 private LocalDateTime timestamp; // 时间戳, 测量时间, 精确到微秒
22
23 @Column(name = "value", nullable = false)
24 private Float value; // 心率变异性值, 单位: 毫秒 (ms), SDNN值, 可用于推测情绪压力
25 }
```

2. 数据传输对象 (DTO)

步数DTO

代码块

```
1 package com.example.health.dto;
2
3 import lombok.Data;
4 import java.time.LocalDateTime;
5
6 @Data
7 public class StepCountDTO {
8     private Integer managerNo;
9     private LocalDateTime startTime;
10    private LocalDateTime endTime;
11    private Integer steps;
12 }
```

心率DTO

```
代码块
1 package com.example.health.dto;
2
3 import lombok.Data;
4 import java.time.LocalDateTime;
5
6 @Data
7 public class HeartRateDTO {
8     private Integer managerNo;
9     private LocalDateTime timestamp;
10    private String value;
11 }
```

睡眠时段DTO

```
代码块
1 package com.example.health.dto;
2
3 import lombok.Data;
4 import java.time.LocalDateTime;
5
6 @Data
7 public class SleepPeriodDTO {
8     /**
9      * 用户ID, 外键
10     */
11     @Column(name = "manager_no")
12     private Long managerNo;
13
14     /**
15      * 开始时间, 精确到微秒
16     */
17     @Column(name = "start_time")
18     private Date startTime;
19
20     /**
21      * 结束时间, 精确到微秒
22     */
23     @Column(name = "end_time")
24     private Date endTime;
25
26     /**
27      * 总睡眠时长, 单位: 分钟, 可为空
28     */
29 }
```

```

29  private Long totalSleepDuration;
30
31  /**
32   * 订单的app来源 0:小程序 1:心知了 2:oppo 3:小米 4: 微信公众号 6:杭州健康通 7:果实
33   * 8:开放平台 9:华为运动健康 10:华为 11:荣泰 16:苹果 17:观照（七院）21:医疗服务平台
34   */
35  private Long source;
36
37  /**
38   * 渠道来源
39   */
40  private String channel;
41
42  /**
43   * 0 未删除 1 删除
44   */
45  @Column(name = "delete_flag")
46  private Long deleteFlag;
47  }

```

睡眠阶段DTO

代码块

```

1  package com.example.health.dto;
2
3  import lombok.Data;
4  import java.time.LocalDateTime;
5
6  @Data
7  public class SleepStageDTO {
8      /**
9       * 用户ID, 外键
10      */
11      @Column(name = "sleep_period_id")
12      private Long sleepPeriodId;
13
14      /**
15       * 开始时间, 精确到微秒
16       */
17      @Column(name = "start_time")
18      private Date startTime;
19

```

```

20  /**
21   * 结束时间，精确到微秒
22   */
23   @Column(name = "end_time")
24   private Date endTime;
25
26  /**
27   * 睡眠类型 0:清醒,1:浅睡,2:深睡,3:REM
28   */
29   private Long stageType;
30
31  /**
32   * 阶段名称，唯一，如“清醒”、“浅睡”、“深睡”、“REM”
33   */
34   private String stageTypeName;
35
36  /**
37   * 订单的app来源 0:小程序 1:心知了 2:oppo 3:小米 4: 微信公众号 6:杭州健康通 7:果实
38   * 8:开放平台 9:华为运动健康 10:华为 11:荣泰 16:苹果 17:观照（七院）21:医疗服务平台
39   */
40   private Long source;
41
42  /**
43   * 渠道来源
44   */
45   private String channel;
46
47  /**
48   * 0 未删除 1 删除
49   */
50   @Column(name = "delete_flag")
51   private Long deleteFlag;
52 }

```

运动DTO

代码块

```

1  package com.example.health.dto;
2
3  import lombok.Data;
4  import java.time.LocalDateTime;
5
6  @Data

```



```
7 public class ExerciseDTO {
8     private Integer managerNo;
9     private Integer exerciseTypeId;
10    private LocalDateTime startTime;
11    private LocalDateTime endTime;
12    private Float distance;
13    private Integer caloriesBurned;
14 }
```

血氧DTO

代码块

```
1 package com.example.health.dto;
2
3 import lombok.Data;
4 import java.time.LocalDateTime;
5
6 @Data
7 public class BloodOxygenDTO {
8     private Integer managerNo;
9     private LocalDateTime timestamp;
10    private Float value;
11 }
```

呼吸率DTO

代码块

```
1 package com.example.health.dto;
2
3 import lombok.Data;
4 import java.time.LocalDateTime;
5
6 @Data
7 public class RespirationRateDTO {
8     private Integer managerNo;
9     private LocalDateTime timestamp;
10    private Float value;
11 }
```

体温DTO

代码块

```
1 package com.example.health.dto;
2
3 import lombok.Data;
4 import java.time.LocalDateTime;
5
6 @Data
7 public class BodyTemperatureDTO {
8     private Integer managerNo;
9     private LocalDateTime timestamp;
10    private Float value;
11 }
```

HRVDTO

代码块

```
1 package com.example.health.dto;
2
3 import lombok.Data;
4 import java.time.LocalDateTime;
5
6 @Data
7 public class HRVDTO {
8     private Integer managerNo;
9     private LocalDateTime timestamp;
10    private Float value;
11 }
```

3. 数据访问层 (Repository)

步数Repository

代码块

```
1 package com.example.health.repository;  
2  
3 import com.example.health.entity.StepCount;  
4 import org.springframework.data.jpa.repository.JpaRepository;  
5  
6 public interface StepCountRepository extends JpaRepository<StepCount, Integer>  
7 {  
8  
9 }
```

心率Repository

代码块

```
1 package com.example.health.repository;  
2  
3 import com.example.health.entity.HeartRate;  
4 import org.springframework.data.jpa.repository.JpaRepository;  
5  
6 public interface HeartRateRepository extends JpaRepository<HeartRate, Integer>  
7 {  
8  
9 }
```

睡眠时段Repository

代码块

```
1 package com.example.health.repository;  
2  
3 import com.example.health.entity.SleepPeriod;  
4 import org.springframework.data.jpa.repository.JpaRepository;  
5  
6 public interface SleepPeriodRepository extends JpaRepository<SleepPeriod,  
7 Integer> {  
8  
9 }
```

睡眠阶段Repository

代码块

```
1 package com.example.health.repository;
2
3 import com.example.health.entity.SleepStage;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface SleepStageRepository extends JpaRepository<SleepStage,
    Integer> {
7 }
```

运动Repository

代码块

```
1 package com.example.health.repository;
2
3 import com.example.health.entity.Exercise;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface ExerciseRepository extends JpaRepository<Exercise, Integer> {
7 }
```

血氧Repository

代码块

```
1 package com.example.health.repository;
2
3 import com.example.health.entity.BloodOxygen;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface BloodOxygenRepository extends JpaRepository<BloodOxygen,
    Integer> {
7 }
```

呼吸率Repository

代码块

```
1 package com.example.health.repository;
```

```
2
3 import com.example.health.entity.RespirationRate;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface RespirationRateRepository extends
  JpaRepository<RespirationRate, Integer> {
7 }
```

体温Repository

代码块

```
1 package com.example.health.repository;
2
3 import com.example.health.entity.BodyTemperature;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface BodyTemperatureRepository extends
  JpaRepository<BodyTemperature, Integer> {
7 }
```

HRVRepository

代码块

```
1 package com.example.health.repository;
2
3 import com.example.health.entity.HRV;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface HRVRepository extends JpaRepository<HRV, Integer> {
7 }
```

睡眠阶段类型Repository

代码块

```
1 package com.example.health.repository;
2
```

```
3 import com.example.health.entity.SleepStageType;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface SleepStageTypeRepository extends
  JpaRepository<SleepStageType, Integer> {
7 }
```

运动类型Repository

代码块

```
1 package com.example.health.repository;
2
3 import com.example.health.entity.ExerciseType;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface ExerciseTypeRepository extends JpaRepository<ExerciseType,
  Integer> {
7 }
```

4. 服务层 (Service)

步数服务

代码块

```
1 package com.example.health.service;
2
3 import com.example.health.dto.StepCountDTO;
4 import com.example.health.entity.StepCount;
5 import com.example.health.entity.User;
6 import com.example.health.repository.StepCountRepository;
7 import com.example.health.repository.UserRepository;
8 import org.springframework.beans.factory.annotation.Autowired;
9 import org.springframework.stereotype.Service;
10
11 @Service
```

```

12 public class StepCountService {
13
14     @Autowired
15     private StepCountRepository stepCountRepository;
16
17     @Autowired
18     private UserRepository userRepository;
19
20     public StepCount saveStepCount(StepCountDTO dto) {
21         User user = userRepository.findById(dto.getmanagerNo())
22             .orElseThrow(() -> new RuntimeException("用户不存在"));
23         StepCount stepCount = new StepCount();
24         stepCount.setUser(user);
25         stepCount.setStartTime(dto.getStartTime());
26         stepCount.setEndTime(dto.getEndTime());
27         stepCount.setSteps(dto.getSteps());
28         return stepCountRepository.save(stepCount);
29     }
30 }

```

心率服务

代码块

```

1 package com.example.health.service;
2
3 import com.example.health.dto.HeartRateDTO;
4 import com.example.health.entity.HeartRate;
5 import com.example.health.entity.User;
6 import com.example.health.repository.HeartRateRepository;
7 import com.example.health.repository.UserRepository;
8 import org.springframework.beans.factory.annotation.Autowired;
9 import org.springframework.stereotype.Service;
10
11 @Service
12 public class HeartRateService {
13
14     @Autowired
15     private HeartRateRepository heartRateRepository;
16
17     @Autowired
18     private UserRepository userRepository;
19
20     public HeartRate saveHeartRate(HeartRateDTO dto) {

```

```

21     User user = userRepository.findById(dto.getmanagerNo())
22         .orElseThrow(() -> new RuntimeException("用户不存在"));
23     HeartRate heartRate = new HeartRate();
24     heartRate.setUser(user);
25     heartRate.setTimestamp(dto.getTimestamp());
26     heartRate.setValue(dto.getValue());
27     return heartRateRepository.save(heartRate);
28 }
29 }

```

睡眠时段服务

代码块

```

1  package com.example.health.service;
2
3  import com.example.health.dto.SleepPeriodDTO;
4  import com.example.health.entity.SleepPeriod;
5  import com.example.health.entity.User;
6  import com.example.health.repository.SleepPeriodRepository;
7  import com.example.health.repository.UserRepository;
8  import org.springframework.beans.factory.annotation.Autowired;
9  import org.springframework.stereotype.Service;
10
11  @Service
12  public class SleepPeriodService {
13
14      @Autowired
15      private SleepPeriodRepository sleepPeriodRepository;
16
17      @Autowired
18      private UserRepository userRepository;
19
20      public SleepPeriod saveSleepPeriod(SleepPeriodDTO dto) {
21          User user = userRepository.findById(dto.getmanagerNo())
22              .orElseThrow(() -> new RuntimeException("用户不存在"));
23          SleepPeriod sleepPeriod = new SleepPeriod();
24          sleepPeriod.setUser(user);
25          sleepPeriod.setStartTime(dto.getStartTime());
26          sleepPeriod.setEndTime(dto.getEndTime());
27          sleepPeriod.setTotalSleepDuration(dto.getTotalSleepDuration());
28          return sleepPeriodRepository.save(sleepPeriod);
29      }
30  }

```


睡眠阶段服务

代码块

```
1  package com.example.health.service;
2
3  import com.example.health.dto.SleepStageDTO;
4  import com.example.health.entity.SleepStage;
5  import com.example.health.entity.SleepPeriod;
6  import com.example.health.entity.SleepStageType;
7  import com.example.health.repository.SleepPeriodRepository;
8  import com.example.health.repository.SleepStageRepository;
9  import com.example.health.repository.SleepStageTypeRepository;
10 import org.springframework.beans.factory.annotation.Autowired;
11 import org.springframework.stereotype.Service;
12
13 @Service
14 public class SleepStageService {
15
16     @Autowired
17     private SleepStageRepository sleepStageRepository;
18
19     @Autowired
20     private SleepPeriodRepository sleepPeriodRepository;
21
22     @Autowired
23     private SleepStageTypeRepository sleepStageTypeRepository;
24
25     public SleepStage saveSleepStage(SleepStageDTO dto) {
26         SleepPeriod sleepPeriod =
27             sleepPeriodRepository.findById(dto.getSleepPeriodId())
28                 .orElseThrow(() -> new RuntimeException("睡眠时段不存在"));
29         SleepStageType stageType =
30             sleepStageTypeRepository.findById(dto.getStageId())
31                 .orElseThrow(() -> new RuntimeException("睡眠阶段类型不存在"));
32         SleepStage sleepStage = new SleepStage();
33         sleepStage.setSleepPeriod(sleepPeriod);
34         sleepStage.setStageType(stageType);
35         sleepStage.setStartTime(dto.getStartTime());
36         sleepStage.setEndTime(dto.getEndTime());
37         return sleepStageRepository.save(sleepStage);
38     }
39 }
```

运动服务

代码块

```
1 package com.example.health.service;
2
3 import com.example.health.dto.ExerciseDTO;
4 import com.example.health.entity.Exercise;
5 import com.example.health.entity.ExerciseType;
6 import com.example.health.entity.User;
7 import com.example.health.repository.ExerciseRepository;
8 import com.example.health.repository.ExerciseTypeRepository;
9 import com.example.health.repository.UserRepository;
10 import org.springframework.beans.factory.annotation.Autowired;
11 import org.springframework.stereotype.Service;
12
13 @Service
14 public class ExerciseService {
15
16     @Autowired
17     private ExerciseRepository exerciseRepository;
18
19     @Autowired
20     private UserRepository userRepository;
21
22     @Autowired
23     private ExerciseTypeRepository exerciseTypeRepository;
24
25     public Exercise saveExercise(ExerciseDTO dto) {
26         User user = userRepository.findById(dto.getmanagerNo())
27             .orElseThrow(() -> new RuntimeException("用户不存在"));
28         ExerciseType exerciseType =
29             exerciseTypeRepository.findById(dto.getExerciseTypeId())
30                 .orElseThrow(() -> new RuntimeException("运动类型不存在"));
31         Exercise exercise = new Exercise();
32         exercise.setUser(user);
33         exercise.setExerciseType(exerciseType);
34         exercise.setStartTime(dto.getStartTime());
35         exercise.setEndTime(dto.getEndTime());
36         exercise.setDistance(dto.getDistance());
37         exercise.setCaloriesBurned(dto.getCaloriesBurned());
38         return exerciseRepository.save(exercise);
39     }
```

血氧服务

代码块

```
1 package com.example.health.service;
2
3 import com.example.health.dto.BloodOxygenDTO;
4 import com.example.health.entity.BloodOxygen;
5 import com.example.health.entity.User;
6 import com.example.health.repository.BloodOxygenRepository;
7 import com.example.health.repository.UserRepository;
8 import org.springframework.beans.factory.annotation.Autowired;
9 import org.springframework.stereotype.Service;
10
11 @Service
12 public class BloodOxygenService {
13
14     @Autowired
15     private BloodOxygenRepository bloodOxygenRepository;
16
17     @Autowired
18     private UserRepository userRepository;
19
20     public BloodOxygen saveBloodOxygen(BloodOxygenDTO dto) {
21         User user = userRepository.findById(dto.getmanagerNo())
22             .orElseThrow(() -> new RuntimeException("用户不存在"));
23         BloodOxygen bloodOxygen = new BloodOxygen();
24         bloodOxygen.setUser(user);
25         bloodOxygen.setTimestamp(dto.getTimestamp());
26         bloodOxygen.setValue(dto.getValue());
27         return bloodOxygenRepository.save(bloodOxygen);
28     }
29 }
```

呼吸率服务

代码块

```
1 package com.example.health.service;
2
3 import com.example.health.dto.RespirationRateDTO;
4 import com.example.health.entity.RespirationRate;
5 import com.example.health.entity.User;
```

```

6  import com.example.health.repository.RespirationRateRepository;
7  import com.example.health.repository.UserRepository;
8  import org.springframework.beans.factory.annotation.Autowired;
9  import org.springframework.stereotype.Service;
10
11  @Service
12  public class RespirationRateService {
13
14      @Autowired
15      private RespirationRateRepository respirationRateRepository;
16
17      @Autowired
18      private UserRepository userRepository;
19
20      public RespirationRate saveRespirationRate(RespirationRateDTO dto) {
21          User user = userRepository.findById(dto.getmanagerNo())
22              .orElseThrow(() -> new RuntimeException("用户不存在"));
23          RespirationRate respirationRate = new RespirationRate();
24          respirationRate.setUser(user);
25          respirationRate.setTimestamp(dto.getTimestamp());
26          respirationRate.setValue(dto.getValue());
27          return respirationRateRepository.save(respirationRate);
28      }
29  }

```

体温服务

代码块

```

1  package com.example.health.service;
2
3  import com.example.health.dto.BodyTemperatureDTO;
4  import com.example.health.entity.BodyTemperature;
5  import com.example.health.entity.User;
6  import com.example.health.repository.BodyTemperatureRepository;
7  import com.example.health.repository.UserRepository;
8  import org.springframework.beans.factory.annotation.Autowired;
9  import org.springframework.stereotype.Service;
10
11  @Service
12  public class BodyTemperatureService {
13
14      @Autowired
15      private BodyTemperatureRepository bodyTemperatureRepository;

```

```

16
17 @Autowired
18 private UserRepository userRepository;
19
20 public BodyTemperature saveBodyTemperature(BodyTemperatureDTO dto) {
21     User user = userRepository.findById(dto.getmanagerNo())
22         .orElseThrow(() -> new RuntimeException("用户不存在"));
23     BodyTemperature bodyTemperature = new BodyTemperature();
24     bodyTemperature.setUser(user);
25     bodyTemperature.setTimestamp(dto.getTimestamp());
26     bodyTemperature.setValue(dto.getValue());
27     return bodyTemperatureRepository.save(bodyTemperature);
28 }
29 }

```

HRV服务

代码块

```

1 package com.example.health.service;
2
3 import com.example.health.dto.HRVDTO;
4 import com.example.health.entity.HRV;
5 import com.example.health.entity.User;
6 import com.example.health.repository.HRVRepository;
7 import com.example.health.repository.UserRepository;
8 import org.springframework.beans.factory.annotation.Autowired;
9 import org.springframework.stereotype.Service;
10
11 @Service
12 public class HRVService {
13
14     @Autowired
15     private HRVRepository hrvRepository;
16
17     @Autowired
18     private UserRepository userRepository;
19
20     public HRV saveHRV(HRVDTO dto) {
21         User user = userRepository.findById(dto.getmanagerNo())
22             .orElseThrow(() -> new RuntimeException("用户不存在"));
23         HRV hrv = new HRV();
24         hrv.setUser(user);
25         hrv.setTimestamp(dto.getTimestamp());

```

```
26     hrv.setValue(dto.getValue());
27     return hrvRepository.save(hrv);
28 }
29 }
```

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密

李亮 5810 2026年02月09日 20:22
注意保密-禁止泄密